Implicitly move from rvalue references in return statements

Document number: P0527R1
Date: 2017-11-08
Project: Programming Language C++, Core Working Group
Reply-to: David Stone: davidmstone@google.com

Introduction

A function return statement will automatically invoke a move rather than a copy in some situations. This proposal slightly extends those situations.

This proposal was approved by EWG in the Kona 2017 meeting.

Return Statements

We already have implicit moves for local values and function parameters in a return statement. The following code compiles just fine:

```
std::unique_ptr<T> f(std::unique_ptr<T> ptr) {
    return ptr;
}
```

However, the following code does not compile

```
std::unique_ptr<T> f(std::unique_ptr<T> && ptr) {
    return ptr;
}
```

Instead, you must type

```
std::unique_ptr<T> f(std::unique_ptr<T> && ptr) {
    return std::move(ptr);
}
```

This extra typing is unnecessary and leads to possible inefficiencies. Consider the generic case:

```
auto f(args...) {
    decltype(auto) result = some_other_function();
    return std::forward<decltype(result)>(result);
}
```

Now we have inhibited return-value optimization (RVO) in the case where `result` is a value and not a reference, but without the `std::forward`, the code does not compile if `result` is an rvalue reference and is move-only.

```
auto f(T && value) {
    // do stuff with value
    return value;
}
```

Using move constructors instead of copy constructors can be critical for writing high-performance C++ code. However, due to the rules of when a move is invoked in a return statement, a user who knows most (but not all) of the rules is likely to accidentally invoke a copy when they mean to invoke a move.

For instance, consider the following two functions:

```
std::string f(std::string x) { return x; }
std::string g(std::string && x) { return x; }
```

The function `f` will invoke the move constructor of `x`, but the function `g` will invoke the copy constructor.

I believe that it is more surprising to the user that a move does not occur in this situation than it would be if a move did occur. The only way to get an rvalue reference to something is if it was bound to a prvalue or if the user explicitly called `std::move` or equivalent. This means that either it is definitely safe (due to being bound to a prvalue) or the user explicitly opted in to moves at some point.

Throw Expressions

Current rules also prevent the compiler from implicitly moving from function parameters in any situation for a throw expression. However, at least one production compiler (clang) does not implement this correctly and implicitly moves from function parameters. This proposal also extends these rules to throw expressions, requiring using a move instead of a copy for local variable and function parameter objects and rvalue references. This proposal also removes the restriction on implicitly moving from a catch-clause parameter in a throw expression. At least one production compiler (gcc) does not implement this correctly and implicitly moves from catch-clause parameters in throw expressions.

This does not cause problems with function parameters and function-scope try-blocks because there is already a rule preventing crossing try-block scope boundaries in implicit moves.

This proposal unifies the treatment of return statements and throw expressions.

Effect On the Standard Library

This proposal is unlikely to have any effect on the standard library, as we typically do not describe function bodies, but rather, function signatures.

Wording

Change in 15.8.3 (class.copy.elision), paragraph 3:

> A movable entity is a non-volatile object or an rvalue reference to a non-volatile type, in either case with automatic storage duration. The underlying type of a movable entity is the type of the object or the referenced type, respectively. In the following copy-initialization contexts, a move operation might be used instead of a copy operation:
>
> - If the expression in a return statement is a (possibly parenthesized) id-expression that names a movable entity ~~an object with automatic storage duration~~ declared in the body or parameter-declaration-clause of the innermost enclosing function or lambda-expression, or
> - if the operand of a throw-expression is a (possibly parenthesized) id-expression that names a movable entity ~~the name of a non-volatile automatic object (other than a function or catch-clause parameter)~~ whose scope does not extend beyond the end of the innermost enclosing try-block (if there is one),
>
> overload resolution to select the constructor for the copy is first performed as if the entity ~~object~~ were designated by an rvalue. If the first overload resolution fails or was not performed, or if the type of the first parameter of the selected constructor is not an rvalue reference to the (possibly cv-qualified) underlying type of the movable entity ~~object's type (possibly cv-qualified)~~, overload resolution is performed again, considering the entity ~~object~~ as an lvalue.

Alternatively, we could replace the two bullets with the following sentence:

> If the operand of a throw expression or the expression in a return statement is a (possibly parenthesized) id-expression that names a movable entity whose scope does not extend beyond the end of the innermost enclosing try-block (if any), or function or lambda-expression, respectively,

Acknowledgements

Thanks to Jens Maurer for helping me draft the wording for this proposal.